PGARA: Profile-Guided Adaptive Register Allocation

Stochastic Dynamic Programming and Game-Theoretic Auctions in an End-to-End Compiler, Without Graph Colouring

Shreyans Chowdry
Reg. No. 24BDS0114
Vellore Institute of Technology
Vellore, India

Samrender Singh Saini
Reg. No. 24BCE2865
Vellore Institute of Technology
Vellore, India

Karthikeya Jami
Reg. No. 24BCE2136
Vellore Institute of Technology
Vellore, India

Abhishek Kumar
Reg. No. 24BCE0585
Vellore Institute of Technology
Vellore, India

Nishanth Kumar
Reg. No. 24BAI0194
Vellore Institute of Technology
Vellore, India

Abstract*—Optimising compilers translate high-level programs into machine code for processors with only a handful of fast registers, and the register allocator, which decides which values live in those registers, is one of the largest single contributors to the quality of the generated code. The allocator has to decide, for every value, whether to keep it in a register or spill it to memory, and the right answer depends on how often each path through the program is actually taken, which the compiler does not know statically. Production allocators colour an interference graph or scan live intervals, ranking spill candidates by static use counts or $10^{\text{loop depth}}$ weights; recent learned allocators train neural policies on large corpora. Static weights are blind to data-dependent branches, and learned policies are costly to train and offer no per-decision explanation. We present PGARA (Profile-Guided Adaptive Register Allocation), which adapts every register decision to the program's measured behaviour. It models the residency of every SSA value as a two-state Markov decision process over the control-flow graph, with transition probabilities measured by profiling. Exact policy iteration yields each value's expected register-residency gain, values sharing a loop-carried φ-web bid as one coalition, and registers are sold in a sealed-bid auction along a maximum-cardinality-search order of the chordal SSA interference graph, refined by best-response dynamics to a pure Nash equilibrium. PGARA is embedded in a complete compiler for a C subset (DFA lexer, LL(1) parser, attribute-grammar semantic analysis, three-address code, SSA optimiser, PG-RISC code generator and cycle-counting simulator) with a web interface that visualises every phase. Across ten benchmarks and five register budgets, all 200 compiled programs produce reference-identical output; at $k{=}4$ registers PGARA lowers the geometric-mean cycle count by 20.5% relative to linear scan, 6.4% relative to loop-depth priority and 2.5% relative to the same engine driven by static branch priors, and cuts dynamic spill traffic by 59% relative to linear scan. Future work will add live-range splitting to the MDP and evaluate the allocator inside a production back end on standard suites.

***Index Terms*—Compiler design, game theory, Markov decision process, profile-guided optimization, register allocation, static single assignment**

I. Introduction

Compilers are the bridge between the abstractions programmers write and the narrow interface that hardware executes. Their structure has been stable for decades (a front end that recognises and checks the language, a middle end that transforms a machine-independent intermediate representation, and a back end that maps it to instructions and registers), but the techniques inside each phase keep evolving, from reusable multi-level infrastructures such as MLIR [1] to optimisation heuristics learned from data [2]–[4]. Static single assignment (SSA) form [5], [6] is now the common currency of this middle end, and the textbooks used to teach the subject treat it as such [7], [8].

Among back-end problems, register allocation has an outsized effect on performance: a value that is not kept in a register costs a store when it is produced and a load each time it is consumed, and on a loop that runs millions of times these accesses dominate execution time. The problem is hard for two reasons. First, the decision is combinatorial, since values that are live at the same time compete for the same k registers. Second, the payoff of each decision is uncertain at compile time, because it depends on which branches the program actually takes. A value used only on an error path that runs once in a thousand iterations is worth far less than one used on the hot path, yet a static analysis sees the two paths as symmetric. Our motivation is to treat this second difficulty as a first-class modelling concern rather than a tuning detail.

Existing allocators fall broadly into three families. Graph-colouring allocators, descending from Chaitin [9] and the priority-based variant of Chow and Hennessy [10], build an interference graph and colour it with k colours, choosing spill candidates by a cost that weights each access by 10^d for loop depth d. Linear-scan allocators [11] trade quality for speed by sweeping live intervals in program order. On SSA form the interference graph is chordal [12], [13], which makes

colouring itself polynomial and moves the difficulty entirely into the spilling decision; combinatorial approaches solve allocation and scheduling jointly to proven optimality [14], [15].

A newer family learns allocation and related decisions from data. MLGO [16] trains policies inside LLVM's production allocator, RL4ReAl [17] learns end-to-end register allocation with reinforcement learning over embeddings such as IR2Vec [18], and environments such as CompilerGym [19] and representations such as ProGraML [20] make these experiments reproducible. In parallel, profile-guided post-link optimisers such as BOLT [21] and Propeller [22] have shown that measured execution profiles yield large gains in code layout, and recent work predicts profiles when none are available [23].

These families leave a gap. Static spill weights encode a fixed belief about branch behaviour (loops iterate about ten times; both arms of a conditional are equally likely), so they mis-rank values precisely on programs with data-dependent control flow, as early studies of static estimators already observed [24], [25]. Learned policies avoid fixed beliefs, but they require large training corpora and expensive training loops [26], and the decision they make for a particular value cannot be traced back to a quantity a compiler engineer can inspect.

Classical colouring formulations also obscure the objective. A colouring is either feasible or not, and the quality of the result emerges indirectly from the order in which nodes are simplified and spilled. Neither family states, for each value, the expected number of cycles that keeping it in a register will save.

We propose PGARA (Profile-Guided Adaptive Register Allocation), which makes that expected saving the central quantity. It is profile-guided because every probability it uses is measured on a training run, and adaptive because the same program receives a different allocation when its measured behaviour changes, without any hand-tuned weights. Each SSA value's residency is a two-state Markov decision process (MDP) over its live range in the control-flow graph [27], [28]; transition probabilities are measured edge frequencies, and exact policy iteration [29] gives the value's expected gain per invocation. Allocation is then a resource-contention game: values that form a loop-carried φ-web bid together as a coalition, in the spirit of cooperative game theory [31], registers are sold in a sealed-bid auction [32] processed along a maximum-cardinality-search (MCS) order [33] of the chordal interference graph, and best-response dynamics settle the result into a pure Nash equilibrium [34], [35]. The approach is apt because each ingredient matches a structural fact of the problem: SSA makes the contention structure chordal, profiles make the uncertainty measurable, and the gain is an explicit, inspectable number.

The advantages are practical as well as conceptual. Policy iteration on a two-state MDP per value converges in about 1.3 improvement rounds on average, so allocation takes milliseconds; every decision can be explained by a bid and an auction round; and the same engine runs unchanged on static priors, which isolates the value of profiling. To evaluate it honestly we built a complete compiler around it, from a formally specified grammar to a cycle-counting simulator, and a web interface that renders every intermediate artefact.

The remainder of the paper is organised as follows. Section II reviews fifteen recent works and the gaps they leave. Section III formalises the language, the front end, the intermediate representations and the optimiser, then develops the MDP, the gain, the coalitions and the auction, with the algorithms and their properties. Section IV describes the toolchain, the parameters and the results on five metrics against three baselines. Section V concludes, and the team's contributions are listed after it.

II. Related Works

We review fifteen works from 2018–2024, grouped by theme. For each we summarise the proposed solution, its methodology and tools, the reported effectiveness and the research gap relative to PGARA.

A. Combinatorial and survey work on allocation

Castañeda Lozano et al. [14] propose Unison, which formulates global register allocation (including spilling, coalescing and live-range splitting) together with instruction scheduling as a single constraint model. Programs are first transformed into linear SSA, the model is solved with a constraint programming solver using problem-specific decomposition, and it is integrated with LLVM for Hexagon, ARM and MIPS. Unison produces provably optimal or near-optimal code for functions of up to about a thousand instructions and improves on LLVM's heuristics for a measurable fraction of them. *Gap:* the objective is a static cost estimate, and solve times grow quickly with function size.

Castañeda Lozano and Schulte [15] survey combinatorial approaches to register allocation and instruction scheduling. They classify integer programming, constraint programming, PBQP and related models by the subproblems covered, by scalability and by the quality of the code produced, and compare reported evaluations across targets. The survey shows that combinatorial methods reach optimality on medium-sized functions but are seldom used in production. *Gap:* almost all surveyed models assume static frequency estimates rather than measured, stochastic behaviour.

B. Learning-based allocation and program representation

VenkataKeerthy et al. [17] present RL4ReAl, a reinforcement-learning register allocator. The interference graph is embedded with IR2Vec features and a hierarchy of agents learns to pick nodes to colour, spill or split, trained with policy-gradient methods inside an LLVM-based gRPC environment for x86 and AArch64. The learned allocator matches or improves on LLVM's greedy allocator on several benchmark suites. *Gap:* training is costly and individual decisions are not interpretable as expected savings.

Trofin et al. [16] introduce MLGO, a framework for integrating trained policies in LLVM. Policies for inlining-for-size and, later, register-allocation eviction are trained offline with reinforcement learning and evolution strategies on corpora of production code and shipped as compiled TensorFlow models. Inlining-for-size achieved several percent size reduction on large binaries. *Gap:* policies generalise from a corpus, not from the profile of the program being compiled.

Cummins et al. [19] present CompilerGym, a set of environments that expose compiler optimisation problems through the OpenAI Gym interface. It wraps LLVM phase ordering, GCC flag tuning and loop-nest generation behind a client-server architecture, with datasets of millions of programs and multiple observation spaces. It made reinforcement-learning experiments on compilers reproducible and fast. *Gap:* it targets pass-level decisions, not per-value register decisions.

Cummins et al. [20] propose ProGraML, a graph representation of programs that combines control, data and call flow, processed by gated graph neural networks. The representation is built from LLVM and XLA IR and evaluated on data-flow analyses and heuristics such as device mapping and thread coarsening. It outperformed sequence-based representations on those tasks. *Gap:* it learns to approximate analyses that PGARA computes exactly.

VenkataKeerthy et al. [18] introduce IR2Vec, distributed embeddings of LLVM IR derived from a learned seed-embedding vocabulary and flow analysis. Embeddings are computed at instruction, function and program level without heavy training, and used for heterogeneous device mapping and thread-coarsening prediction with competitive accuracy. *Gap:* embeddings capture syntax and flow, not execution frequencies.

Jain et al. [36] propose POSET-RL, which orders optimisation passes with reinforcement learning. A partially ordered set of pass sub-sequences derived from LLVM's O-levels defines the action space, and a deep Q-network trained on IR2Vec states optimises size and execution time. It reduced code size and runtime relative to the standard pipeline on SPEC and MiBench. *Gap:* it operates above the allocator and treats it as a black box.

Haj-Ali et al. [37] present NeuroVectorizer, which learns vectorisation and interleaving factors with deep reinforcement learning. Loops are embedded with code2vec and a policy chooses pragmas that Clang/LLVM applies, with rewards measured by execution time. It outperformed the baseline cost model on a range of loop kernels. *Gap:* measured rewards drive training, but the decision is not grounded in a model of the program's behaviour.

C. Surveys of machine learning in compilers

Wang and O'Boyle [2] survey machine learning in compiler optimisation. They organise the field by feature engineering, learning models and deployment, covering search-based tuning, predictive modelling and deep learning across many published systems, and show consistent gains over hand-written heuristics. *Gap:* register allocation is barely represented, and the role of measured profiles inside heuristics is left open.

Ashouri et al. [3] survey compiler autotuning with machine learning, with an emphasis on optimisation selection and phase ordering. They classify works by learning model, features and prediction type and summarise the tools used, from Milepost-GCC to LLVM-based frameworks, reporting the speed-ups achieved. *Gap:* the focus is on choosing passes, not on resource assignment inside a pass.

D. Profile-guided optimisation

Panchenko et al. [21] present BOLT, a post-link optimiser that uses sampled hardware profiles. It rewrites binaries using LLVM's MC infrastructure and reorders basic blocks and functions according to Linux `perf` profiles collected in production. It gave substantial speed-ups on large data-centre applications already built with feedback-directed optimisation. *Gap:* it uses profiles for layout only; register decisions are fixed before it runs.

Shen et al. [22] present Propeller, a profile-guided relinking optimiser for warehouse-scale applications. It integrates profile-driven basic-block layout into the build through distributed, incremental relinking with the LLVM toolchain, avoiding BOLT's monolithic binary rewrite. It achieved comparable speed-ups with much lower memory and build-time cost. *Gap:* as with BOLT, the profile does not reach allocation.

Rotem and Cummins [23] predict branch probabilities statically so that profile-guided optimisation can run without profiles. A gradient-boosted model is trained on features of branches in a large corpus of profiled code and its predictions are fed into LLVM's PGO pipeline. A substantial share of the benefit of real profiles was recovered. *Gap:* predicted probabilities feed existing static heuristics rather than an allocation model that uses them directly.

E. Infrastructure and large language models

Lattner et al. [1] introduce MLIR, a multi-level compiler infrastructure with extensible dialects, regions and SSA-based IR that is shared across domains. It is implemented in C++ within the LLVM project and used to build compilers for machine learning, hardware design and more. It reduced the cost of building domain-specific compilers. *Gap:* MLIR lowers to LLVM's allocators and does not change how registers are assigned.

Cummins et al. [38] train a large language model to optimise LLVM assembly for code size. A 7B-parameter transformer is trained from scratch to predict pass lists and optimised IR, with auxiliary tasks that improve its understanding of the code. It outperformed the compiler's built-in pass ordering on held-out programs with no extra compilations. *Gap:* its outputs are not guaranteed correct and give no account of the decisions made.

F. Summary of gaps

Across these works, measured execution frequencies drive layout but not allocation, learned allocators require heavy training and are opaque, and combinatorial allocators optimise

static cost models. PGARA addresses the intersection: an allocator whose objective is the expected saving under measured probabilities, solved exactly and cheaply, with every decision explainable as a bid.

III. Proposed Work

A. Problem formulation

Let a function in SSA form have values V, a control-flow graph (CFG) $\mathcal{C} = (B, E, b_0)$ and a register file of k registers. An allocation is a map $\rho : V \to \{0, \dots, k-1\} \cup \{\bot\}$ in which $\rho(v) = \bot$ means that v lives in its stack slot; it is feasible when no two interfering values share a register. Executing the program on an input induces a random walk over $\mathcal{C}$, and the cost of the allocation is the expected number of cycles spent on loads, stores and copies caused by values without a register. We seek

$$\rho^\star \;=\; \arg\max_{\rho \text{ feasible}} \sum_{v:\, \rho(v) \neq \bot} G(v), \tag{1}$$

where $G(v)$ is the expected cycles saved by giving v a register, computed under transition probabilities measured by profiling.

Two features make (1) tractable without colouring. On strict SSA form the interference graph is chordal [12], so the values competing at any program point form a clique that an MCS order exposes one at a time. And the gain of a single value can be computed exactly by stochastic dynamic programming over its live range, because the residency of one value is a small MDP whose only uncertainty is the branch behaviour of the CFG. Both observations are developed below, after the front end and middle end that produce the SSA program.

B. System architecture

Fig. 1 shows the architecture. The front end turns MiniC source text into a type-checked abstract syntax tree (AST) and symbol table; the middle end lowers it to three-address code (TAC), builds the CFG, constructs SSA and optimises it; a reference interpreter executes the optimised SSA on a training input and records edge counts; the PGARA engine converts those counts into gains, coalitions and an auctioned allocation; and the back end leaves SSA, emits PG-RISC code and runs it on a cycle-counting simulator on a separate test input.

Every stage reports its result into a single JSON document, which the web interface renders as token streams, AST, symbol tables, TAC, SSA, CFG, interference graph, auction rounds and target code (dashed arrows in Fig. 1). Three baseline allocators share the same pipeline and code generator, so differences in the measured cycles are attributable to allocation decisions alone.

C. Front end: language, grammar and parsing

Lexical analysis: MiniC has `int` scalars and one-dimensional arrays, array parameters passed by reference, functions, `if`/`else`, `while`, `for`, `break`, `continue`, `print` and `input()`, the arithmetic, relational and short-circuit logical operators, and the compound forms `+=`, `-=`, `*=`, `++`, `--`. The lexer is a deterministic finite automaton applied under the maximal-munch rule: from position i it returns the longest prefix w of the remaining input with $\delta^*(q_0, w) \in F$, so `<=` is one token and `a+++b` is `a ++ + b`. Keywords are recognised as identifiers and reclassified by table lookup. An invalid character is reported with its line and column and skipped (panic mode), so later errors are still found in the same run.

Context-free grammar: The syntax is the context-free grammar

$$G = (N, \Sigma, P, S), \quad |N| = 41,\ |P| = 94,\ S = \textit{Program}, \tag{2}$$

written directly in LL(1) form. Left recursion is removed by the standard tail transformation and precedence is encoded by one nonterminal per level, for example

$$\begin{aligned}
\textit{Add} &\to \textit{Mul AddRest}\\
\textit{AddRest} &\to \texttt{+}\ \textit{Mul AddRest} \mid \texttt{-}\ \textit{Mul AddRest} \mid \varepsilon\\
\textit{Stmt} &\to \texttt{if}\ \ \texttt{(}\ \textit{Expr}\ \texttt{)}\ \textit{Stmt ElsePart}\\
&\quad \mid \texttt{while}\ \ \texttt{(}\ \textit{Expr}\ \texttt{)}\ \textit{Stmt} \mid \cdots\\
\textit{ElsePart} &\to \texttt{else}\ \textit{Stmt} \mid \varepsilon.
\end{aligned}$$

The precedence chain, from loosest to tightest binding, is *Or*, *And*, *Eq*, *Rel*, *Add*, *Mul*, *Unary*, *Primary*, giving the usual C precedence and left associativity.

FIRST, FOLLOW and the LL(1) property: For $\alpha \in (N \cup \Sigma)^*$ and $A \in N$,

$$\begin{aligned}
\mathrm{FIRST}(\alpha) &= \{a \in \Sigma \mid \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\},\\
\mathrm{FOLLOW}(A) &= \{a \in \Sigma \cup \{\$\} \mid S\$ \Rightarrow^* \beta A a \gamma\}.
\end{aligned} \tag{3}$$

Both are computed as least fixpoints of the usual inclusion constraints. G is LL(1) iff for every pair of alternatives $A \to \alpha \mid \beta$

$$\mathrm{FIRST}\big(\alpha\,\mathrm{FOLLOW}(A)\big) \cap \mathrm{FIRST}\big(\beta\,\mathrm{FOLLOW}(A)\big) = \emptyset. \tag{4}$$

The toolchain computes (3), builds the predictive table and checks (4) mechanically. The single conflict is the cell (*ElsePart*, `else`): `else` is in $\mathrm{FIRST}(\texttt{else}\ \textit{Stmt})$ and, because *ElsePart* can end a *Stmt*, also in $\mathrm{FOLLOW}(\textit{ElsePart})$. This is the dangling-else ambiguity, resolved by preferring $\textit{ElsePart} \to \texttt{else}\ \textit{Stmt}$ so that an `else` binds to the nearest unmatched `if`. Every other cell holds at most one production, so G is LL(1) modulo this resolution.

Relation to LR parsing: Every LL(1) grammar is also LR(1) [39], [40], so an LR(1) or LALR parser generator would accept G unchanged apart from the same dangling-else conflict, which appears there as a shift/reduce conflict on `else` and is conventionally resolved in favour of shift, binding `else` the same way. We chose top-down predictive parsing because a recursive-descent parser is a direct transcription of the grammar (one procedure per nonterminal), errors can be reported in terms of the construct being parsed, and the GUI can show the predictive table, which students can check against (3)–(4). The parser recovers from syntax errors in panic mode, discarding tokens until one in the FOLLOW set of the current construct (for statements, `;` or `}`) is found.

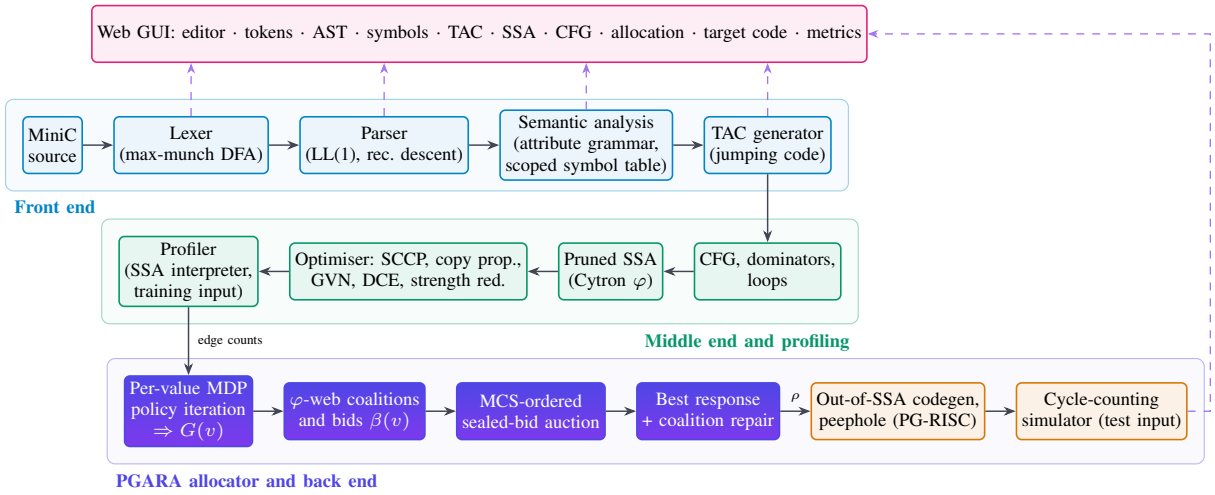


Fig. 1. Architecture of the PGARA compiler: front end (blue), middle end and profiling (green), the PGARA allocation engine (indigo) and back end (amber). Dashed violet arrows show phase reports consumed by the web interface (pink).

D. Semantic analysis as an attribute grammar

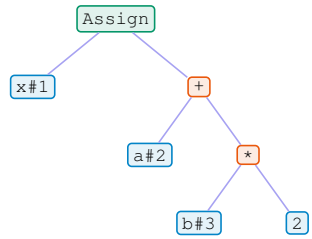
Semantic analysis is specified as an L-attributed grammar [41] and implemented as a traversal of the AST. Each node carries an inherited environment Γ (a stack of scopes, each a hash map from names to symbols) and a synthesised type $\tau \in \{\texttt{int}, \texttt{int[]}, \texttt{void}\}$. Typical rules are

$$
\begin{aligned}
&\mathit{Add} \to \mathit{Mul}\ \mathit{AddRest}: \\
&\quad \mathit{AddRest}.in = \mathit{Mul}.node, \quad \mathit{Add}.node = \mathit{AddRest}.out \\
&\mathit{AddRest} \to + \ \mathit{Mul}\ \mathit{AddRest}_1: \\
&\quad \mathit{AddRest}_1.in = \mathrm{Bin}(+, \mathit{AddRest}.in, \mathit{Mul}.node), \\
&\quad \mathit{AddRest}.out = \mathit{AddRest}_1.out \\
&\mathit{AddRest} \to \varepsilon: \quad \mathit{AddRest}.out = \mathit{AddRest}.in \\
&E \to E_1\ \mathit{op}\ E_2: \quad E.\tau = \texttt{int} \text{ iff } E_1.\tau = E_2.\tau = \texttt{int} \\
&\mathit{Primary} \to \texttt{ID}: \quad \mathit{Primary}.\tau = \Gamma(\texttt{ID}).\tau \text{ (error if unbound)}
\end{aligned}
\tag{5}
$$

The inherited attribute in restores left associativity that the LL(1) transformation removed from the syntax. Fig. 2 shows the flow of attributes on a small statement. Declarations extend Γ in the innermost scope and reject redeclaration in the same scope; uses resolve outward through enclosing scopes. Functions are collected in a first pass, so calls may precede definitions. The analyser checks arity and argument kinds, array indexing and assignment, use of `void` results, `break`/`continue` outside loops and missing returns. Every symbol receives a unique IR name $x\#n$, so shadowed variables never collide in later phases.

E. Three-address code and control flow

The AST is lowered by a syntax-directed translation into three-address code whose instructions have the forms $x = y\ \mathit{op}\ z$, $x = \mathit{op}\ y$, $x = y$, $x = a[i]$, $a[i] = y$, `goto` L, `if` x `goto` L_t `else` L_f, `param`, `call`, `return`, `print` and `input`. Conditions are translated as jumping code [8]: with true and false targets t, f,



Attribute	Flow on `x = a + b * 2;`
Γ (inherited)	passed down to every leaf; resolves `x`, `a`, `b`
τ (synthesised)	leaves `int`; `*`, `+` give `int`; Assign checks
in (inherited)	builds `a + (b*2)` left-associatively
unique name	`x#1`, `a#2`, `b#3` used by IR generation

Fig. 2. Attributed AST of a simple assignment, coloured as in the GUI: statements green, expressions orange, leaves blue. The environment flows down, types flow up, and each identifier is bound to a unique IR name.

$$
\begin{aligned}
\mathcal{J}[\![B_1 \,\texttt{\&\&}\, B_2]\!](t,f) &= \mathcal{J}[\![B_1]\!](L,f)\ ;\ L{:}\ \mathcal{J}[\![B_2]\!](t,f), \\
\mathcal{J}[\![B_1 \,\texttt{||}\, B_2]\!](t,f) &= \mathcal{J}[\![B_1]\!](t,L)\ ;\ L{:}\ \mathcal{J}[\![B_2]\!](t,f), \\
\mathcal{J}[\![\texttt{!}\,B]\!](t,f) &= \mathcal{J}[\![B]\!](f,t),
\end{aligned}
\tag{6}
$$

with L a fresh label, so short-circuit evaluation is exact and no Boolean value is materialised unless it is stored. Basic blocks are maximal sequences with a single entry and a single exit. The CFG is cleaned of unreachable blocks and its critical edges are split, so that copies inserted later on an edge have a block of their own. Dominators are computed by the Cooper–Harvey–Kennedy iterative algorithm over reverse postorder [42], natural loops are identified from back edges $n \to h$ with $h \,\mathrm{dom}\, n$, and each block gets a loop depth. Fig. 3 shows the TAC and optimised SSA the compiler produces for a small loop.

TAC

```
entry: n = input()
       s = 0
       i = 0
for1:  t2 = i < n
       if t2 goto body2
            else endfor4
body2: t3 = i % 3
       t4 = t3 == 0
       if t4 goto and7
            else endif6
and7:  t5 = i > 0
       if t5 goto then5
            else endif6
then5: s = s + i
endif6:
step3: i = i + 1
       goto for1
endfor4:
       print s
       return 0
```

Optimised SSA

```
entry: n = input()
       goto for1
for1:  i.2 = phi(0, i.3)
       s.2 = phi(0, s.4)
       if i.2 < n goto body2
            else endfor4
body2: t3 = i.2 % 3
       if t3 == 0 goto and7
            else endif6
and7:  if i.2 > 0 goto then5
            else endif6
then5: s.3 = s.2 + i.2
endif6:
       s.4 = phi(s.2, s.2, s.3)
       i.3 = i.2 + 1
       goto for1
endfor4:
       print s.2
       return 0
```

Fig. 3. Compiler output for `for(i=0;i<n;i++) if(i%3==0 && i>0) s+=i;`. Jumping code realises `&&`; after SSA and optimisation, comparisons are fused into branches and constants into φ arguments (split critical-edge blocks omitted).

F. SSA construction and optimisation

Liveness is the least fixpoint of

$$\begin{aligned} \mathit{LiveIn}(b) &= \mathit{use}(b) \cup \big(\mathit{LiveOut}(b) \setminus \mathit{def}(b)\big), \\ \mathit{LiveOut}(b) &= \textstyle\bigcup_{s \in \mathit{succ}(b)} \mathit{LiveIn}(s), \end{aligned} \tag{7}$$

solved by a worklist over reverse postorder, with φ-uses attributed to the corresponding predecessor edge. SSA is built by the method of Cytron et al. [5]: for each variable x with definition blocks D_x, a φ is placed at every block of the iterated dominance frontier $DF^{+}(D_x)$ at which x is live on entry (pruned SSA), and variables are renamed by a walk of the dominator tree. A read with no reaching definition becomes the constant 0, which matches MiniC's zero-initialisation of locals.

The optimiser iterates, to a fixpoint or ten rounds, sparse conditional constant propagation over the lattice $\top \sqsupset c \sqsupset \bot$ with executable-edge tracking (SCCP) [43], copy propagation, algebraic simplification and strength reduction (for example $x \cdot 2^m \Rightarrow x \ll m$, $x + 0 \Rightarrow x$), global value numbering scoped by the dominator tree, dead-code elimination over SSA def-use chains and the merging of straight-line blocks. It then splits critical edges again and fuses each comparison with the branch that consumes it into a compare-branch instruction. Semantic preservation is checked on every benchmark by executing the SSA program before and after optimisation and comparing outputs.

G. Profiling and transition probabilities

The optimised SSA program is executed by a reference interpreter with bounds-checked memory on a training input. The interpreter counts every CFG edge $n(b, s)$, and the profiled transition matrix is

$$P_{\mathrm{prof}}(b, s) = \frac{n(b, s)}{\sum_{s' \in \mathit{succ}(b)} n(b, s')}, \tag{8}$$

falling back to the static prior for blocks never reached in training. The Static-Prior variant uses the classical loop heuristic [25] instead: if exactly one successor leaves a loop containing b,

$$P_{\mathrm{stat}}(b, s) = \begin{cases} 0.1 & s \text{ exits the loop}, \\ 0.9 & \text{otherwise}, \end{cases} \tag{9}$$

and otherwise $P_{\mathrm{stat}}(b, s) = 1/|\mathit{succ}(b)|$. Fig. 4 contrasts the two on the inner loop of the rare-branch benchmark, where an error path is taken in about 1% of iterations but the static prior sees a fair coin.

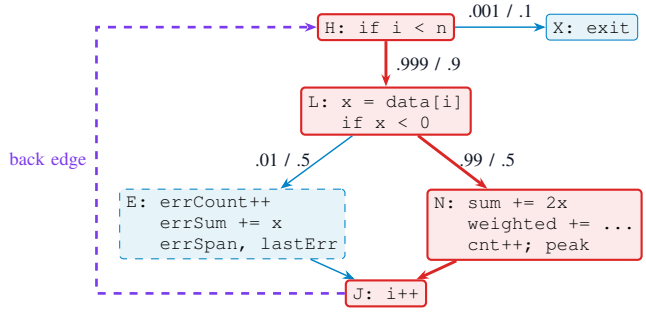


Fig. 4. Inner loop of the rare-branch benchmark. Edge labels give $P_{\mathrm{prof}}/P_{\mathrm{stat}}$. Red marks the hot path found by profiling and blue the rarely executed blocks: the error block E runs in 1% of iterations, whereas the static prior assumes 50%.

H. The residency MDP of a value

Definition 1 (Residency MDP): For an SSA value v with live range $L_v \subseteq B$, the residency MDP has states (b, s) with $b \in L_v$ and $s \in \{\mathsf{R}, \mathsf{M}\}$ (control is in block b and v is in a register or in its slot). In state R the actions are *stay* and *spill* (cost c_{st}); in state M they are *stay* and *reload* (cost c_{ld}). Control moves from b to $b' \in L_v$ with probability $P(b, b')$ when $v \in \mathit{LiveIn}(b')$; leaving the live range is absorbing with value 0.

Let $u_v(b)$, $d_v(b)$ and $f_v(b)$ be the numbers of uses, definitions and *forwardable* uses (a use immediately after the definition, which the peephole pass turns into a register move when v is spilled) of v in b, and $c_v(b)$ the number of calls in b across which v is live. The per-block rewards are

$$\begin{aligned} r_{\mathsf{R}}(b) &= \sigma\,\big(u_v(b) + d_v(b) + f_v(b)\big) - c_v(b)\,(c_{\mathrm{st}} + c_{\mathrm{ld}}), \\ r_{\mathsf{M}}(b) &= -\big(u_v(b)\,c_{\mathrm{ld}} + d_v(b)\,c_{\mathrm{st}} + f_v(b)\,c_{\mathrm{mv}}\big), \end{aligned} \tag{10}$$

with σ a small bonus for a register access, c_{mv} the cost of a forwarded move and an extra $-c_{\mathrm{ld}}$ in the definition block of a parameter, which arrives in its slot. The optimal values satisfy the Bellman equations [27], [30]

$$\begin{aligned} V(b, \mathsf{R}) &= r_{\mathsf{R}}(b) + \max\big\{\bar{V}_{\mathsf{R}}(b),\ -c_{\mathrm{st}} + \bar{V}_{\mathsf{M}}(b)\big\}, \\ V(b, \mathsf{M}) &= r_{\mathsf{M}}(b) + \max\big\{\bar{V}_{\mathsf{M}}(b),\ -c_{\mathrm{ld}} + \bar{V}_{\mathsf{R}}(b)\big\}, \\ \bar{V}_s(b) &= \textstyle\sum_{b' \in L_v} P(b, b')\,V(b', s), \end{aligned} \tag{11}$$

Fig. 5 draws the two states. Because a live range is left with probability one under every policy of a terminating program, the chain restricted to L_v is transient and (11) is solved

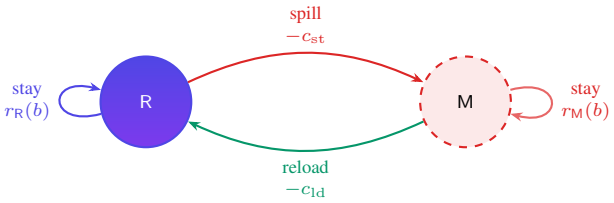


Fig. 5. The two-state residency MDP replicated over each block b of a value's live range. In either state control moves to b' with probability $P(b, b')$; leaving L_v is absorbing with value 0. Rewards follow (10).

undiscounted, as a total-reward MDP. For a fixed policy π, policy evaluation is the sparse linear system

$$\big(I - P_\pi\big)\, V_\pi = r_\pi, \tag{12}$$

where P_π is substochastic and transient, so $I - P_\pi$ is a non-singular M-matrix and Gaussian elimination without pivoting is stable. Howard's policy iteration [29] alternates (12) with greedy improvement and terminates in a finite number of rounds with the exact optimum; we keep value iteration as an independent cross-check in the test suite. If the restricted chain were ever not transient, the solver would fall back to a discount factor $\gamma = 0.999$.

I. Expected gain of a value

An auction loser does not choose its policy: it is forced to stay in M throughout. Let V_{mem} be the value of that forced policy, obtained from (12) with $\pi \equiv \textit{stay}$. The per-entry gain is the difference between entering the live range with a register under the optimal policy and entering it without one, $g(v) = V^*(\mathit{def}(v), \mathsf{R}) - V_{\mathrm{mem}}(\mathit{def}(v), \mathsf{M})$. The live range is entered once for each execution of the definition block, whose expected count per invocation ϕ solves the Markov-chain flow equations

$$\phi = e_{b_0} + P^{\top}\phi, \tag{13}$$

so the expected register-residency gain of v per invocation is

$$G(v) = \phi\big(\mathit{def}(v)\big) \cdot \Big(V^*\big(\mathit{def}(v), \mathsf{R}\big) - V_{\mathrm{mem}}\big(\mathit{def}(v), \mathsf{M}\big)\Big). \tag{14}$$

Measuring against V_{mem} rather than against $V^*(\cdot, \mathsf{M})$ matters: an optimal policy that starts in M may reload at once, which would cap every gain at roughly c_{ld} and hide the difference between hot and cold values. $G(v)$ is the quantity a compiler engineer would ask for (expected cycles saved), and the GUI shows it for every value. In the rare-branch example of Fig. 4, the profiled MDP rates the hot-path accumulators `cnt.3` and `weighted.3` at $G = 20{,}300$ and the error-path counters `errSum.3` and `errCount.3` at $G = 180$, a factor of 113. Under the static prior (9) all four receive the same gain, 202.5, because the two arms of the branch look alike; Static-Prior consequently keeps the error-path value `lastErr` in a register and spills `cnt` and `weighted`, whereas PGARA does the opposite.

J. Coalitions on φ-webs

A loop-carried variable is a chain of SSA values joined by φ-functions. If they are allocated independently, the auction may grant a register to one version and not the next, and every traversal of the back edge then pays a copy through memory. We therefore form coalitions by union-find over φ edges, merging the argument a and result p of a φ in block h along edge (b, h) only when the edge is *hot*:

$$\phi(b)\, P(b, h) \;\geq\; \theta \cdot \max\big\{\phi(\mathit{def}(a)),\ \phi(h)\big\}, \qquad \theta = 0.25. \tag{15}$$

A loop-carried variable thus forms one coalition around its back edge, while a copy made once on loop entry, or on a rare path, does not drag a cold value into a hot coalition. Every member of coalition C bids the coalition's total gain,

$$\beta(v) = \sum_{w \in C(v)} G(w), \tag{16}$$

so the coalition wins or loses as a unit. This is the transferable-utility view of cooperative game theory [31]: the coalition's value is what its members achieve together, and (16) lets members with small individual gains (for example the version of `i` after the increment, $G = 12{,}288$) inherit the bid of the whole web ($\beta = 94{,}223$).

K. Sealed-bid auction on the chordal interference graph

The interference graph $\mathcal{I} = (V, A)$ joins two values when one is live at the definition of the other. On strict SSA $\mathcal{I}$ is chordal [12]; we compute a maximum cardinality search order σ [33] and verify on every function that its reverse is a perfect elimination ordering (PEO), which held for all functions in our suite. In a chordal graph visited in MCS order, the already-visited neighbours of each vertex form a clique, so when v arrives its already-visited neighbours together with v are a clique Q_v, a set of values that are all simultaneously live.

The auction (Algorithm 2) visits values in order σ. If v has fewer than k already-winning neighbours, it wins outright. Otherwise Q_v exceeds the register budget and a sealed-bid round awards the k registers of that clique to the k highest bids:

$$W(Q_v) = \arg\max_{S \subseteq Q_v,\ |S| \leq k} \sum_{w \in S} \beta(w), \tag{17}$$

which for a single capacity constraint is solved exactly by sorting. Ties are broken by σ, so the result is deterministic. Losers are spilled to slots that are shared by every member of a φ-web, so that spilled φ copies become no-ops. Registers are then bound along σ, choosing for each winner a register not held by any winning neighbour and preferring one already held by a member of its web, so the φ copy disappears. Fig. 6 shows a real auction round from the rare-branch benchmark at $k = 4$.

L. Best-response dynamics and equilibrium

The auction decides each clique locally, so a loser can end up with a register that none of its neighbours actually uses (for example because a rival that beat it later lost elsewhere).

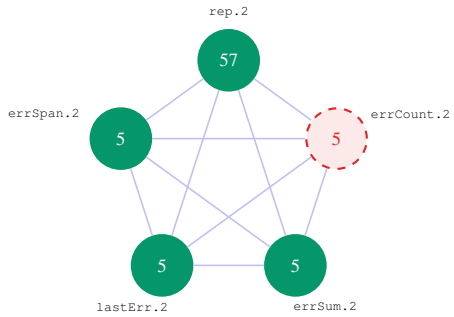


Fig. 6. An auction round at $k = 4$ in the rare-branch benchmark: when `errCount.2` arrives in MCS order its clique has five members. Node labels are bids β; the four highest win (green), and the tie at 5 is broken by MCS order, so `errCount.2` is spilled (red, dashed). The GUI uses the same colours for auction winners and losers.

Algorithm 1 Profile-driven gains and coalition bids

Require: SSA function with CFG $\mathcal{C}$, edge counts n (or none)
Ensure: gains G, bids β, coalitions C

$P \leftarrow$ (8) if n given, else (9)
$\phi \leftarrow$ solve $(I - P^{\top})\phi = e_{b_0}$ (13)
for all $v \in V$ **do**
  build rewards $r_{\mathsf{R}}, r_{\mathsf{M}}$ on L_v (10)
  $\pi \leftarrow$ *stay* in every state
  repeat
    $V_\pi \leftarrow$ solve $(I - P_\pi)V_\pi = r_\pi$ (12)
    $\pi' \leftarrow$ greedy in (11) w.r.t. V_π
    if $\pi' = \pi$ **then break**; $\pi \leftarrow \pi'$
  until false
  $V_{\text{mem}} \leftarrow$ evaluate $\pi \equiv$ *stay* from M
  $g \leftarrow V_\pi(def(v), \mathsf{R}) - V_{\text{mem}}(def(v), \mathsf{M})$
  $G(v) \leftarrow \phi(def(v)) \cdot g$ (14)
end for
$C \leftarrow$ union-find over φ edges satisfying (15)
$\beta(v) \leftarrow \sum_{w \in C(v)} G(w)$ for all v (16)
return G, β, C

We model the result as a game whose players are coalitions, whose strategies are a register for each member or memory, and whose payoff is β when all members hold registers and 0 otherwise, subject to the non-interference constraint. A spilled coalition C has a profitable unilateral deviation if

$$\beta(C) > 0 \ \wedge \ \forall v \in C : \ \{0, \ldots, k{-}1\} \setminus \rho\big(\mathrm{adj}(v)\big) \neq \emptyset. \quad (18)$$

Best-response dynamics (Algorithm 3) let deviating coalitions take free registers, highest bid first, until none can.

Splitting a coalition between registers and memory costs a copy through memory on each hot φ edge that joins a winner and a loser. When that expected penalty is at least the winners' own gain,

$$\sum_{\substack{(b,h)\ \varphi\text{-edge} \\ \text{split}}} \phi(b)\, P(b,h)\, c_{(b,h)} \ \geq \sum_{w \in C,\ \rho(w) \neq \perp} G(w), \quad (19)$$

with $c_{(b,h)}$ equal to c_{ld} or c_{st} depending on the direction of the copy, the winners concede and the whole coalition leaves

Algorithm 2 MCS-ordered sealed-bid auction and register binding

Require: interference graph $\mathcal{I}$, bids β, k, webs
Ensure: allocation ρ

$\sigma \leftarrow$ MaximumCardinalitySearch$(\mathcal{I})$; assert reverse(σ) is a PEO
$win \leftarrow \emptyset$
for all v in order σ **do**
  $R \leftarrow \{w \in \mathrm{adj}(v) \mid w \in win\}$
  if $|R| < k$ **then**
    $win \leftarrow win \cup \{v\}$
  else
    $Q \leftarrow R \cup \{v\}$ {a clique}
    $S \leftarrow$ top-k of Q by $(\beta, -\sigma)$ (17)
    $win \leftarrow (win \setminus Q) \cup S$
  end if
end for
for all $v \in win$ in order σ **do**
  $F \leftarrow \{0..k{-}1\} \setminus \{\rho(w) \mid w \in \mathrm{adj}(v)\}$
  $\rho(v) \leftarrow$ a register of F held by v's web, else $\min F$
end for
return ρ (values without a register are spilled)

the game, after which best response runs again to reuse the freed registers.

Proposition 1: Algorithm 3 terminates after at most $|V|$ deviations, and its result is a pure Nash equilibrium of the coalition game: no spilled coalition satisfies (18), and no winning coalition can raise its payoff by deviating.

Proof sketch. A deviation moves a whole coalition from memory into registers that are free with respect to all of its neighbours, without evicting anyone, so the set of allocated values grows strictly; hence at most $|V|$ deviations occur. Repair only removes values, and the final best-response pass again ends when (18) fails for every spilled coalition. A winning coalition already receives its maximal payoff $\beta(C)$, and changing to another free register leaves the payoff unchanged, so no player has a strictly profitable deviation. □

The equilibrium need not maximise (1) globally, since the register game is a congestion game [34] with many equilibria. The auction is what steers the dynamics towards the high-gain equilibrium: best response only fills registers that the auction left free.

M. Back end: leaving SSA and code generation

Fig. 8 summarises the allocation flow and its hand-off to the back end. The target, PG-RISC, is a load/store RISC with allocatable registers $r_0, \ldots, r_{k-1}$, three scratch registers t_0–t_2 for spilled operands, a return-value register and a frame pointer. The frame holds parameters, spill slots (one per φ-web), caller-save slots and local arrays. φ-functions are replaced by parallel copies on the (already split) incoming edges, sequentialised with cycles broken through t_2; copies between values that share a register or a slot vanish. Registers live across a call are saved and restored around it. A peephole

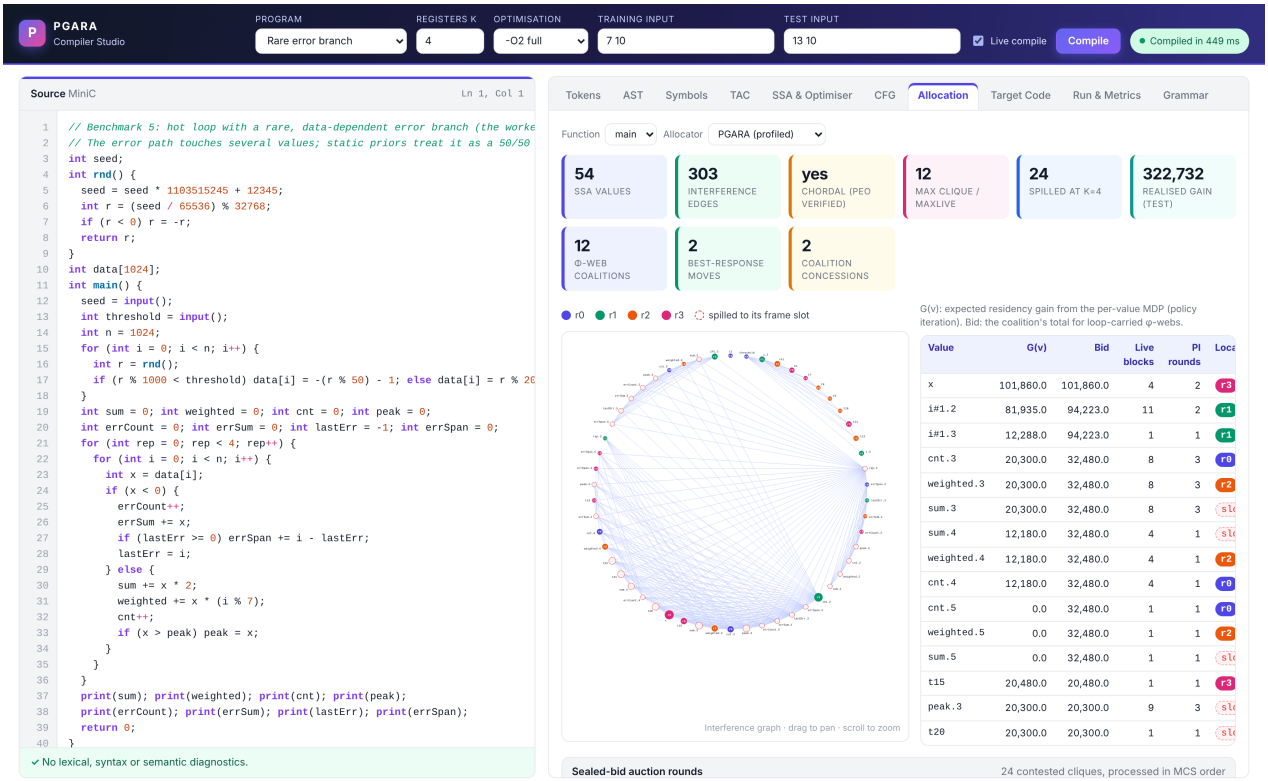


Fig. 7. The PGARA Compiler Studio on the rare-branch benchmark at $k = 4$. Left: the MiniC editor with syntax colouring and diagnostics. Right: the Allocation view with the interference graph (one colour per register, red dashed outline for spilled values), each value's gain $G(v)$, coalition bid, live-range size, policy-iteration rounds and register, and the auction rounds in MCS order with winners in green and losers in red.

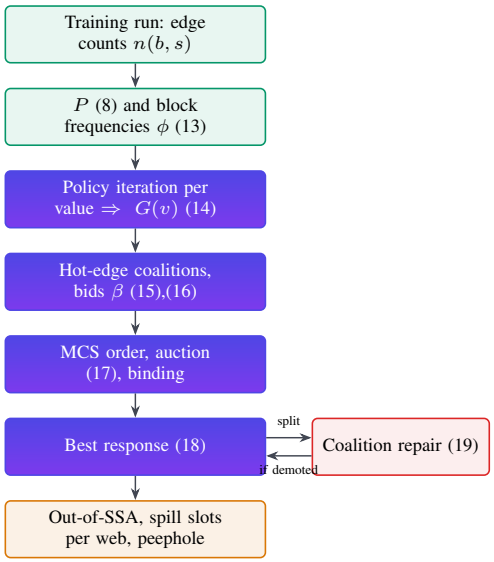


Fig. 8. Flow of the PGARA allocator, from profile to target code.

pass threads jumps, removes jumps to the next instruction and forwards a store to an immediately following load of the same slot, turning the load into a move or deleting it, which is the forwarding that $f_v(b)$ in (10) anticipates.

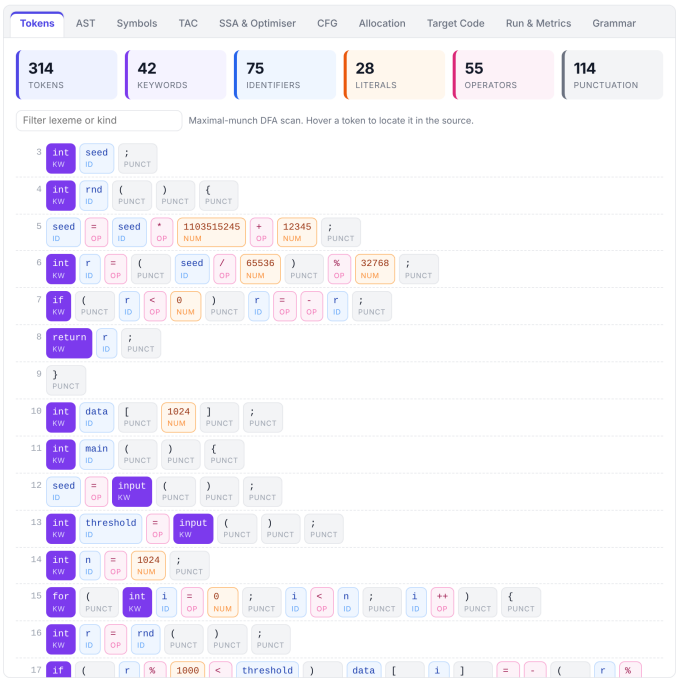
N. Modules and the team's contribution

The work divides into five modules, each owned by one team member (the full list is given in the contributions section). The core pipeline module provides the lexer, the grammar and its LL(1) verification, the parser and AST, and TAC generation (Shreyans Chowdry). The semantic module provides the scoped symbol table, the attribute evaluation and diagnostics (Samrender Singh Saini). The optimisation and code-generation module provides SSA, the optimiser, the PGARA allocator and PG-RISC code generation (Karthikeya Jami). The runtime and evaluation module provides the interpreter, the simulator, the test suite and the benchmark harness (Abhishek Kumar). The interface module provides the web GUI, the visualisations and the documentation (Nishanth Kumar).

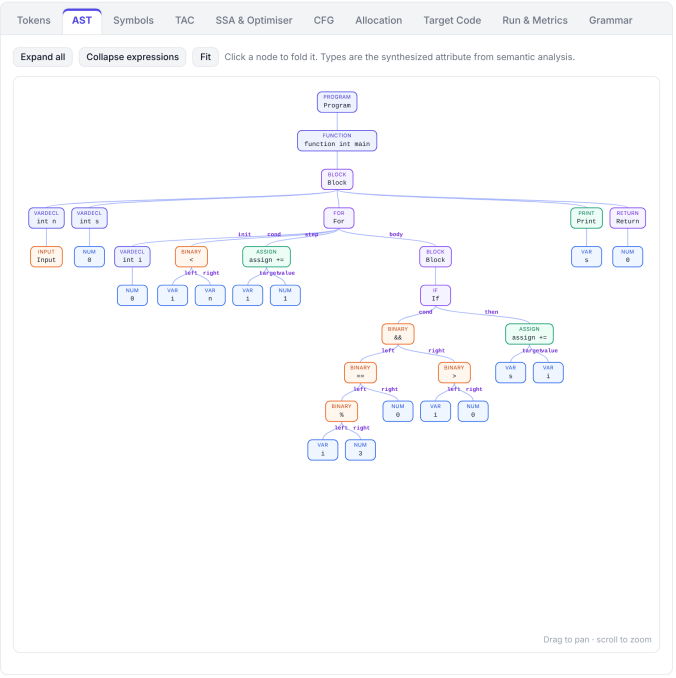
IV. Implementation and Result Analysis

A. Tools and setup

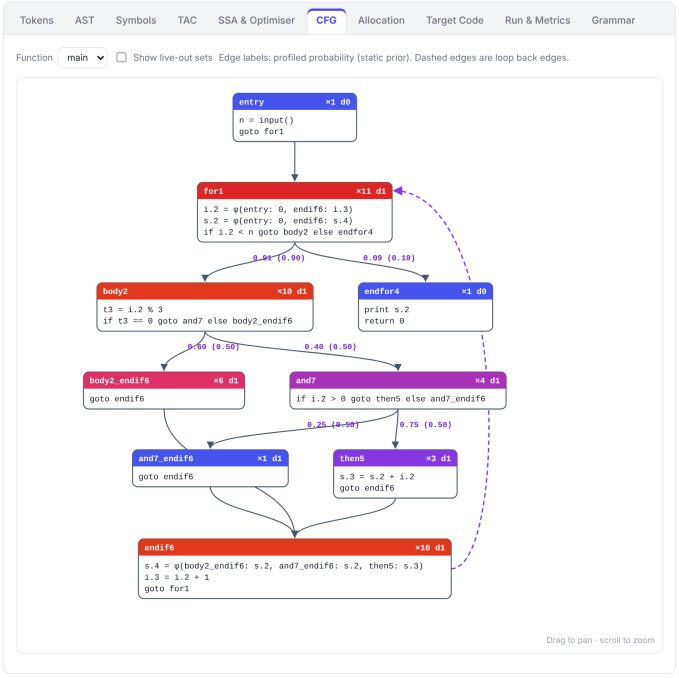
The compiler is written in Python 3 using only the standard library, so it runs on any machine with a recent Python installation and no package manager [44]. A small HTTP server from the standard library serves the web interface and a JSON compilation endpoint; the interface is plain HTML, CSS and JavaScript with hand-drawn SVG for trees and graphs, using a colour code that is consistent across views (token kinds, AST node categories, one colour per register, red for spills, green



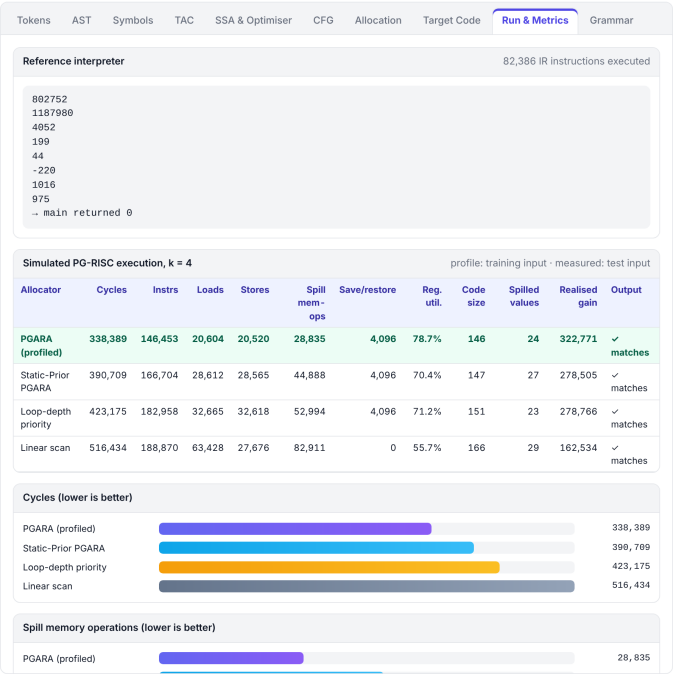
(a) Tokens: the DFA's token stream, coloured by kind



(b) AST: declarations, control flow, statements, expressions and leaves



(c) CFG: block headers coloured by execution frequency, edges labelled P_{prof} (P_{stat})



(d) Run & Metrics: reference output and the four allocators compared

Fig. 9. Further views of the PGARA Compiler Studio. (a) and (d) show the rare-branch benchmark; (b) and (c) show the loop of Fig. 3 compiled with training input 10.

for auction winners, and a blue-to-red heat scale for block frequencies), so that the same colours used in the figures of this paper appear on screen. The GUI is started with a single start-up script, which launches the server on port 8765 and opens the browser, and the benchmark study is run with one command-line invocation that writes JSON and CSV results.

A `unittest` suite of 23 tests covers the lexer, grammar, parser, semantic checks, SSA invariants (single assignment and dominance of uses), optimiser semantic preservation, chordality, agreement of policy and value iteration, a hand-computed gain, auction capacity, the Nash property and end-to-end runs of all allocators for $k \in \{2, 3, 4, 6\}$; all tests pass.

Algorithm 3 Best-response dynamics with coalition repair

Require: ρ from Algorithm 2, coalitions C, bids β, gains G
Ensure: equilibrium allocation ρ

```
BestResponse(ρ)
for all coalitions C split between registers and memory do
    if (19) holds for C then
        ρ(w) ← ⊥ for every w ∈ C
    end if
end for
if anything was demoted then BestResponse(ρ)
return ρ

procedure BestResponse(ρ)
repeat
    changed ← false
    for all spilled coalitions C by decreasing β do
        if every v ∈ C has a free register (18) then
            assign them, preferring registers of web partners
            changed ← true
        end if
    end for
until not changed
```

The interface was exercised end to end in headless Chromium through Playwright [45], which also produced the screenshots in Figs. 7 and 9.

Each benchmark is compiled at the full optimisation level, profiled on a training input and measured on a different test input, for every allocator and $k \in \{3, 4, 5, 6, 8\}$: 200 configurations. Every simulated run is compared with the reference interpreter's output and all 200 match. Experiments ran on a Linux container with Python 3.13; because the simulator counts cycles deterministically, the results do not depend on the host.

B. Parameters and benchmarks

Table I lists the parameters. The cost model matches the simulator's latencies, so the gains are expressed in the same cycles that are measured. The suite (Table II) spans straight-line code, regular loops, recursion and programs whose control flow depends on the data: a rare error branch, an early-exit search, a switch-like dispatch chain and a token scanner. Most programs generate their data with a linear congruential generator seeded from the input, so training and test inputs exercise the same code with different data.

C. Baselines and metrics

We compare four allocators that share the front end, optimiser, interference graph and code generator: (i) PGARA with profiled probabilities; (ii) *Static-Prior* PGARA, the same engine with (9); (iii) *Loop-depth priority*, in the style of Chow and Hennessy [10], which weights each access by 10^{depth} and runs the same auction and binding; and (iv) *Linear scan* [11] over live intervals in layout order, with register hints for φ partners. The proposal's comparison with LLVM's native allocator was replaced by (iii) and (iv) because PG-RISC is not an LLVM target; this keeps every other factor identical.

We report five metrics: (1) simulated *cycles* on the test input; (2) dynamic *spill memory operations*, the loads and stores of spill slots executed; (3) *register utilisation*, $1 -$ spill memops/value accesses, the fraction of operand accesses served from registers; (4) static *code size* in instructions, together with the number of spilled values; and (5) *compile cost*, the policy-iteration rounds and allocation time of Table II.

TABLE I
PARAMETER SETUP

Parameter	Value
Register budgets k	3, 4, 5, 6, 8
Load / store cost c_{ld} / c_{st} (cycles)	4 / 2
Register-access bonus σ, forwarded move c_{mv}	1 / 1
ALU / MUL / DIV latency	1 / 3 / 12
Taken / not-taken branch, call / return	2 / 1, 3 / 3
Discount γ	1 (fallback 0.999)
Static loop-stay probability	0.9
Hot-edge threshold θ	0.25
Optimiser rounds (max)	10
Interpreter / simulator step limits	$3 \cdot 10^6$ / $1.2 \cdot 10^7$
Training $\neq$ test input	all benchmarks

TABLE II
BENCHMARKS: SSA VALUES, φS, LARGEST INTERFERENCE CLIQUE ω, MEAN AND MAXIMUM POLICY-ITERATION ROUNDS, AND TIME TO RUN ALL FOUR ALLOCATORS ($k = 4$)

Benchmark	$\lvert V\rvert$	φ	ω	PI_{mean}	$\text{PI}_{\max}$	ms
straightline	39	0	11	1.00	1	2.6
arraysum	38	16	9	1.29	3	6.2
matmul	51	10	8	1.27	3	5.9
recursion	39	4	7	1.28	3	4.4
rarebranch	62	30	12	1.26	3	9.4
earlyexit	41	17	10	1.29	3	8.0
dispatch	116	55	15	1.17	7	21.7
scanner	89	49	11	1.15	4	19.8
sort	53	16	10	1.32	3	11.9
collatz	30	16	9	1.53	3	9.6

D. Execution time

Fig. 10 shows cycles at $k = 4$, normalised to linear scan. PGARA is fastest or tied on six of ten programs and never more than 1.7% behind the best allocator. The largest gains are on programs with skewed, data-dependent branches: on rare-branch PGARA needs 34.5% fewer cycles than linear scan and 13.4% fewer than Static-Prior, because the static prior treats the error path as a coin flip and spends registers on error counters, while the profile assigns them to the values of the hot path (Fig. 4). Dispatch shows the largest difference from loop-depth priority (22%), because every case of the dispatch chain sits at the same depth and depth weighting cannot tell the common cases from the rare ones. On regular loops (matmul, sort, collatz) the profile and the static prior agree, and PGARA coincides with Static-Prior. Straight-line code favours linear scan by 1.4%: with a single block there is no uncertainty to

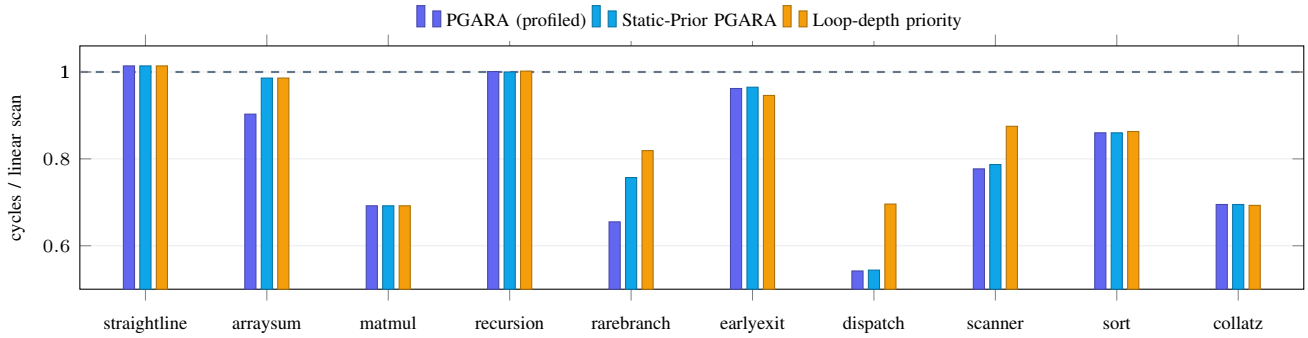


Fig. 10. Simulated cycles at $k = 4$ on the test inputs, normalised to linear scan (dashed grey line, 1.0). Lower is better. Colours match the allocator colours used in the GUI's Run & Metrics view.

TABLE III
CYCLES (THOUSANDS) AND DYNAMIC SPILL MEMORY OPERATIONS AT $k = 4$

	Cycles ($\times 10^3$)				Spill memops			
Bench.	MG	SP	LD	LS	MG	SP	LD	LS
straight.	0.29	0.29	0.29	**0.28**	54	54	54	**45**
arraysum	**311**	340	340	345	**24.7k**	36.9k	36.9k	32.8k
matmul	**566**	**566**	**566**	818	**54.3k**	**54.3k**	**54.3k**	121k
recursion	202	**202**	202	202	181	221	**179**	1343
rarebr.	**338**	391	423	516	**28.8k**	44.9k	53.0k	82.9k
earlyexit	651	652	**640**	676	25.0k	25.2k	**25.0k**	25.2k
dispatch	**128**	129	165	236	**6.4k**	**6.4k**	17.1k	38.8k
scanner	**412**	418	464	531	**5.3k**	6.0k	21.2k	54.4k
sort	**850**	**850**	853	988	71.6k	71.6k	**69.5k**	101k
collatz	655	655	**653**	942	4.8k	4.8k	**4.1k**	76.6k

MG: PGARA; SP: Static-Prior; LD: loop-depth; LS: linear scan. Bold: best.

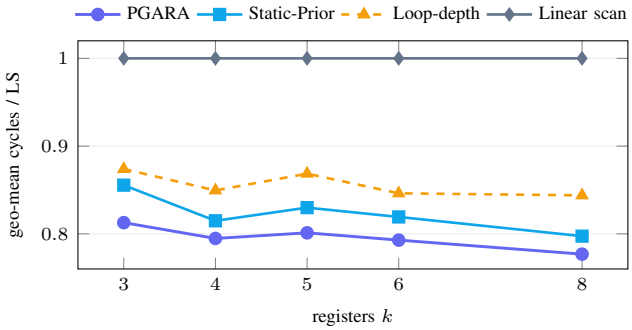


Fig. 11. Geometric-mean cycles over the suite, normalised to linear scan, as the register budget varies.

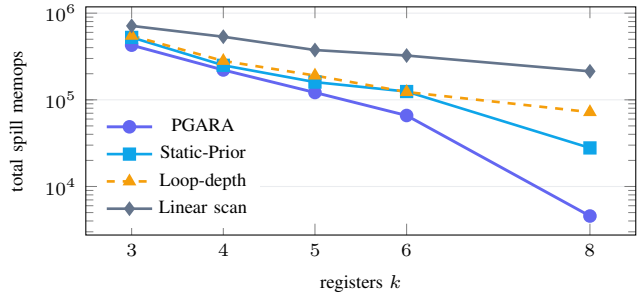


Fig. 12. Dynamic spill loads and stores summed over the suite (log scale).

exploit, and linear scan's interval order happens to fit that block slightly better.

Across register budgets (Fig. 11) PGARA has the lowest geometric-mean cycle count at every k, 19–22% below linear scan, 2.5–5.0% below Static-Prior and 6.3–7.9% below loop-depth priority. Its advantage over Static-Prior is concentrated in the data-dependent benchmarks and is not uniform: on scanner at $k = 5$ and dispatch at $k = 6$ the profiled variant is 4.3% and 2.1% slower, because a value that is cold on average but hot in a short phase is under-rated by average-case probabilities; loop-depth priority is up to 2% faster on earlyexit at $k = 4, 5$ and slightly faster on matmul at $k = 5, 6$. Results are not strictly monotonic in k for every program, because each extra register that holds a value across a call adds caller-save traffic.

E. Spill traffic and register utilisation

Dynamic spill traffic (Fig. 12) is where the expected-gain objective shows most directly, because (14) is precisely the expected spill traffic avoided. Summed over the suite, PGARA executes 59% fewer spill memory operations than linear scan at $k = 4$, 12% fewer than Static-Prior and 21% fewer than loop-depth priority, and at $k = 8$ it nearly eliminates spill traffic (4.6k operations against 27.9k, 72.2k and 213k). Register utilisation (Fig. 13) follows: the mean fraction of operand accesses served from registers rises from 77% at $k = 3$ to 98% at $k = 8$ for PGARA, against 69% to 87% for linear scan. Note that the number of *static* spilled values can be higher for PGARA than for loop-depth priority (191 against 159 at $k = 4$): PGARA deliberately spills many cold values in order to keep the few hot ones, which is the intended behaviour of a gain-maximising allocator.

F. Code size and compile cost

Table IV gives static code size summed over the suite. The three auction-based allocators produce code of similar size, 9–17% smaller than linear scan, because shared web slots and register hints remove most φ copies, and fewer spilled operands need explicit loads and stores. Code size is not the objective of PGARA, and Static-Prior is marginally smaller

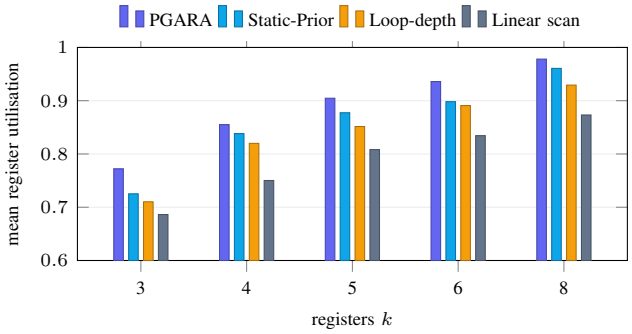


Fig. 13. Mean register utilisation (fraction of operand accesses served from registers) over the suite.

TABLE IV
STATIC CODE SIZE (INSTRUCTIONS) SUMMED OVER THE SUITE

k	PGARA	Static-Prior	Loop-depth	Linear scan
3	**1486**	1496	1511	1661
4	1443	**1433**	1439	1604
5	1395	**1385**	1392	1573
6	1351	**1341**	1360	1541
8	**1257**	1260	1297	1520

at $k = 4$–6. Compile cost is low (Table II): policy iteration needs 1.26 improvement rounds on average over all values and configurations, with a maximum of seven, and running all four allocators on the largest program (dispatch, 116 SSA values) takes about 22 ms in pure Python. Every interference graph in the study was verified chordal, with the largest clique at 15 values.

G. Sensitivity to the training input

A profile-driven allocator is only as good as its profile. Table V repeats the three most data-dependent benchmarks at $k = 4$ with a deliberately unrepresentative training input: on rare-branch the error rate in training is 60% instead of 1%; on dispatch and scanner the training input shifts the case mix. With a mismatched profile PGARA falls behind Static-Prior by 2–10%, though it remains ahead of loop-depth priority and linear scan on all three, only narrowly so on scanner. The training input must therefore resemble production behaviour, as with any profile-guided optimisation [21], [24], and Static-Prior is the right fallback when no such input exists.

H. Comparison with related approaches

Table VI positions PGARA against the classical allocators it was measured against and against the learned allocator RL4ReAl [17], which we compare qualitatively because it targets LLVM's back ends.

I. Threats to validity

The benchmarks are small kernels written for this study, and the threshold θ was tuned on the same suite (from the candidates 0.1, 0.25 and 0.5). Cycles come from a simulator with a fixed latency table rather than from hardware, and

TABLE V
CYCLES ($\times 10^3$, $k = 4$) WITH REPRESENTATIVE AND MISMATCHED TRAINING INPUTS

	PGARA				
Benchmark	repr.	mism.	Static	Loop-d.	Lin. scan
rarebranch	**338.4**	398.4	390.7	423.2	516.4
dispatch	**128.1**	136.7	128.6	164.6	236.4
scanner	**412.3**	459.9	417.8	464.2	530.7

TABLE VI
QUALITATIVE COMPARISON OF ALLOCATION APPROACHES

Approach	Decision basis	Uses measured profile	Training	Explainable
Linear scan [11]	interval order	no	none	partly
Loop-depth priority [10]	10^d weights	no	none	yes
RL4ReAl [17]	learned policy	indirectly	heavy	no
PGARA (this work)	expected gain $G(v)$, auction	yes	none	yes (bids)

PG-RISC has no caches, so the memory cost is a constant. The MDP charges a value for its whole live range, while the optimal policy may move it between register and memory inside the range; the code generator does not split live ranges, so that part of the policy is computed but not realised.

V. CONCLUSION AND FUTURE WORK

PGARA allocates registers adaptively, guided by measured profiles, by expected gain: a per-value Markov decision process over the live range, solved exactly with measured branch probabilities, followed by a sealed-bid auction among φ-web coalitions on the chordal SSA interference graph and best-response dynamics to a Nash equilibrium. Existing allocators either rank values by static weights that cannot see data-dependent branches or learn opaque policies from large corpora.

The method makes the expected saving of each decision an explicit number, needs no training, and costs milliseconds. It is embedded in a complete compiler, from an LL(1) grammar and attribute-grammar semantic analysis through TAC, SSA and optimisation to PG-RISC code and a simulator, with a web interface that shows every phase.

On ten benchmarks and five register budgets, all 200 compiled programs match the reference output. At $k = 4$, PGARA reduces geometric-mean cycles by 20.5% relative to linear scan, 6.4% relative to loop-depth priority and 2.5% relative to static priors, and executes 59% fewer spill memory operations than linear scan, with the largest benefits on programs with skewed, data-dependent control flow.

Future work will extend the MDP's policy into live-range splitting in the code generator so that values can move between register and memory inside a range, replace whole-program averages with context-sensitive or phase-aware profiles to address the cases where the profile misleads, port the engine to

a production back end such as LLVM to evaluate it on standard suites and real hardware, and combine measured profiles with learned profile prediction [23] when no representative input is available.

Author Contributions

1) **Shreyans Chowdry** (24BDS0114): core pipeline design, lexical and syntax analysis, AST construction and intermediate code generation.
2) **Samrender Singh Saini** (24BCE2865): semantic analysis, scope resolution and symbol table management.
3) **Karthikeya Jami** (24BCE2136): code optimisation engine and target assembly and machine code generation.
4) **Abhishek Kumar** (24BCE0585): runtime environment, test suite and benchmarking.
5) **Nishanth Kumar** (24BAI0194): web GUI, pipeline visualisation interface and documentation.

All authors contributed to the design of the PGARA allocator and to the writing of this manuscript.

Acknowledgements

The authors thank their course faculty for guidance on recasting the allocator around stochastic dynamic programming and game theory.

Funding

This work received no external funding.

Competing Interests

The authors declare no competing interests.

Data and Code Availability

The compiler, the web interface, the benchmark programs and the raw results (JSON and CSV for all 200 configurations and the sensitivity study) are available from the authors on request.

References

[1] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[2] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," *Proc. IEEE*, vol. 106, no. 11, pp. 1879–1901, 2018.

[3] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," *ACM Comput. Surv.*, vol. 51, no. 5, art. 96, 2018.

[4] H. Leather and C. Cummins, "Machine learning in compilers: Past, present and future," in *Proc. Forum for Specification and Design Languages (FDL)*, 2020.

[5] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, pp. 451–490, 1991.

[6] F. Rastello and F. Bouchez Tichadou, Eds., *SSA-based Compiler Design*. Cham, Switzerland: Springer, 2022.

[7] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[8] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[9] G. J. Chaitin, "Register allocation & spilling via graph coloring," in *Proc. SIGPLAN Symp. Compiler Construction*, 1982, pp. 98–105.

[10] F. C. Chow and J. L. Hennessy, "The priority-based coloring approach to register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 12, no. 4, pp. 501–536, 1990.

[11] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 21, no. 5, pp. 895–913, 1999.

[12] S. Hack, D. Grund, and G. Goos, "Register allocation for programs in SSA-form," in *Proc. Int. Conf. Compiler Construction (CC)*, LNCS 3923, 2006, pp. 247–262.

[13] S. Hack and G. Goos, "Optimal register allocation for SSA-form programs in polynomial time," *Inf. Process. Lett.*, vol. 98, no. 4, pp. 150–155, 2006.

[14] R. Castañeda Lozano, M. Carlsson, G. Hjort Blindell, and C. Schulte, "Combinatorial register allocation and instruction scheduling," *ACM Trans. Program. Lang. Syst.*, vol. 41, no. 3, art. 17, 2019.

[15] R. Castañeda Lozano and C. Schulte, "Survey on combinatorial register allocation and instruction scheduling," *ACM Comput. Surv.*, vol. 52, no. 3, art. 62, 2019.

[16] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "MLGO: A machine learning guided compiler optimizations framework," arXiv:2101.04808, 2021.

[17] S. VenkataKeerthy, S. Jain, A. Kundu, R. Aggarwal, A. Cohen, and R. Upadrasta, "RL4ReAl: Reinforcement learning for register allocation," in *Proc. ACM SIGPLAN Int. Conf. Compiler Construction (CC)*, 2023, pp. 133–144.

[18] S. VenkataKeerthy, R. Aggarwal, S. Jain, M. S. Desarkar, R. Upadrasta, and Y. N. Srikant, "IR2Vec: LLVM IR based scalable program embeddings," *ACM Trans. Archit. Code Optim.*, vol. 17, no. 4, art. 32, 2020.

[19] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "CompilerGym: Robust, performant compiler optimization environments for AI research," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2022, pp. 92–105.

[20] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. P. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in *Proc. Int. Conf. Machine Learning (ICML)*, PMLR 139, 2021, pp. 2244–2253.

[21] M. Panchenko, R. Auler, B. Nell, and G. Ottoni, "BOLT: A practical binary optimizer for data centers and beyond," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2019, pp. 2–14.

[22] H. Shen, K. Pszeniczny, R. Lavaee, S. Kumar, S. Tallam, and X. D. Li, "Propeller: A profile guided, relinking optimizer for warehouse-scale applications," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[23] N. Rotem and C. Cummins, "Profile guided optimization without profiles: A machine learning approach," arXiv:2112.14679, 2021.

[24] T. A. Wagner, V. Maverick, S. L. Graham, and M. A. Harrison, "Accurate static estimators for program optimization," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 1994, pp. 85–96.

[25] Y. Wu and J. R. Larus, "Static branch frequency and program profile analysis," in *Proc. 27th Annu. Int. Symp. Microarchitecture (MICRO)*, 1994, pp. 1–11.

[26] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[27] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: Wiley, 1994.

[28] D. P. Bertsekas, *Reinforcement Learning and Optimal Control*. Belmont, MA, USA: Athena Scientific, 2019.

[29] R. A. Howard, *Dynamic Programming and Markov Processes*. Cambridge, MA, USA: MIT Press, 1960.

[30] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 1957.

[31] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games II*, Annals of Mathematics Studies 28. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.

[32] W. Vickrey, "Counterspeculation, auctions, and competitive sealed tenders," *J. Finance*, vol. 16, no. 1, pp. 8–37, 1961.

[33] R. E. Tarjan and M. Yannakakis, "Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs," *SIAM J. Comput.*, vol. 13, no. 3, pp. 566–579, 1984.

[34] R. W. Rosenthal, "A class of games possessing pure-strategy Nash equilibria," *Int. J. Game Theory*, vol. 2, no. 1, pp. 65–67, 1973.
[35] N. Nisan, T. Roughgarden, É. Tardos, and V. V. Vazirani, Eds., *Algorithmic Game Theory*. Cambridge, U.K.: Cambridge Univ. Press, 2007.
[36] S. Jain, Y. Andaluri, S. VenkataKeerthy, and R. Upadrasta, "POSET-RL: Phase ordering for optimizing size and execution time using reinforcement learning," in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2022, pp. 121–131.
[37] A. Haj-Ali, N. K. Ahmed, T. Willke, Y. S. Shao, K. Asanovic, and I. Stoica, "NeuroVectorizer: End-to-end vectorization with deep reinforcement learning," in *Proc. IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2020, pp. 242–255.
[38] C. Cummins, V. Seeker, D. Grubisic, M. Elhoushi, Y. Liang, B. Rozière, J. Gehring, F. Gloeckle, K. Hazelwood, G. Synnaeve, and H. Leather, "Large language models for compiler optimization," arXiv:2309.07062, 2023.
[39] D. E. Knuth, "On the translation of languages from left to right," *Inf. Control*, vol. 8, no. 6, pp. 607–639, 1965.
[40] P. M. Lewis and R. E. Stearns, "Syntax-directed transduction," *J. ACM*, vol. 15, no. 3, pp. 465–488, 1968.
[41] D. E. Knuth, "Semantics of context-free languages," *Math. Syst. Theory*, vol. 2, no. 2, pp. 127–145, 1968.
[42] K. D. Cooper, T. J. Harvey, and K. Kennedy, "A simple, fast dominance algorithm," Rice Univ., Houston, TX, USA, Tech. Rep., 2001.
[43] M. N. Wegman and F. K. Zadeck, "Constant propagation with conditional branches," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 2, pp. 181–210, 1991.
[44] Python Software Foundation, "Python 3 documentation," 2024. [Online]. Available: https://docs.python.org/3/
[45] Microsoft, "Playwright: Fast and reliable end-to-end testing for modern web apps," 2024. [Online]. Available: https://playwright.dev/